Print copy of official Microsoft Learn documentation

Source: https://learn.microsoft.com/en-us/powershell/scripting/overview?view=powershell-7.6
Exported: 2026-10-14. Source metadata date: 2026-08-12T00:00:00Z.

Microsoft and documentation contributors. Article wording and code for the requested version are retained; web navigation is removed and print formatting is added. This locally generated PDF is not a Microsoft-published PDF book or a complete product manual. Source-page attribution and notices apply.

Documentation license and terms: Microsoft Learn terms; .NET documentation: CC BY 4.0 license. Check the source for specific attribution or exceptions.

What is PowerShell?

PowerShell is a cross-platform task automation solution made up of a command-line shell, a scripting language, and a configuration management framework. PowerShell runs on Windows, Linux, and macOS.

Command-line Shell

PowerShell is a modern command shell that includes the best features of other popular shells. Unlike most shells that only accept and return text, PowerShell accepts and returns .NET objects. The shell includes the following features:

- Robust command-line history
- Tab completion and command prediction (See about_PSReadLine)
- Supports command and parameter aliases
- Pipeline for chaining commands
- In-console help system, similar to Unix `man` pages

Scripting language

As a scripting language, PowerShell is commonly used for automating the management of systems. It's also used to build, test, and deploy solutions, often in CI/CD environments. PowerShell is built on the .NET Common Language Runtime (CLR). All inputs and outputs are .NET objects. No need to parse text output to extract information from output. The PowerShell scripting language includes the following features:

- Extensible through functions, classes, scripts, and modules
- Extensible formatting system for easy output
- Extensible type system for creating dynamic types
- Built-in support for common data formats like CSV, JSON, and XML

Automation platform

The extensible nature of PowerShell provides an ecosystem of PowerShell modules to deploy and manage almost any technology you work with. For example:

Microsoft modules

- Azure
- Windows
- Exchange
- SQL

Third-party modules

- AWS
- VMware
- Oracle Cloud

Configuration management

PowerShell Desired State Configuration (DSC) is a management framework in PowerShell that enables you to manage your enterprise infrastructure with configuration as code. With DSC, you can:

- Create declarative configurations and custom scripts for repeatable deployments
- Enforce configuration settings and report on configuration drift
- Deploy configuration using push or pull models

Monad Manifesto

Jeffrey Snover, the inventor of PowerShell, wrote the Monad Manifesto to explain his vision for PowerShell and how it would change the way we manage systems. Use the following link to download a copy of the Monad Manifesto.

This PDF file is a version of the original Monad Manifesto, which articulated the long-term vision and started the development effort that became PowerShell. PowerShell has delivered on many of the elements described in this document.

Next steps

Getting started

Are you new to PowerShell and don't know where to start? Take a look at these resources.

- Install PowerShell
- Discover PowerShell
- PowerShell 101
- Microsoft Virtual Academy videos
- PowerShell Learn modules

PowerShell in action

Take a look at how PowerShell is being used in different scenarios and on different platforms.

- PowerShell remoting over SSH

- Getting started with Azure PowerShell
- Building a CI/CD pipeline with DSC
- Managing Microsoft Exchange